

HiDiTingV100 文件系统

使用指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档主要介绍 HiDiTingV100 的文件系统相关内容，包括接口实现机制与使用说明。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危

符号	说明
	害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 概述.....1

2 系统接口.....3

2.1 VFS 3

2.1.1 概述..... 3

2.1.2 开发流程..... 3

2.1.2.1 VFS 结构..... 3

2.1.2.2 VFS 介绍..... 4

2.1.2.3 功能..... 6

2.2 YAFFS 文件系统..... 7

2.2.1 概述..... 7

2.2.2 开发指导..... 7

2.2.2.1 接口说明..... 8

2.2.2.2 开发流程..... 9

2.2.3 注意事项..... 10

3 镜像制作与烧录.....12

3.1 概述..... 12

3.2 YAFFS 文件系统镜像制作与烧录..... 12

3.2.1 概述..... 13

3.2.2 开发指导..... 13

3.2.3 编译指导..... 15

3.2.4 烧录指导..... 16

插图目录

图 1-1 文件系统架构 1

图 2-1 VFS 文件目录结构 4

图 2-2 VFS 对象之间的关系 5

表格目录

表 2-1 posix 接口函数列表..... 6

表 2-2 YAFFS 文件系统默认分区..... 8

表 2-3 接口说明 8

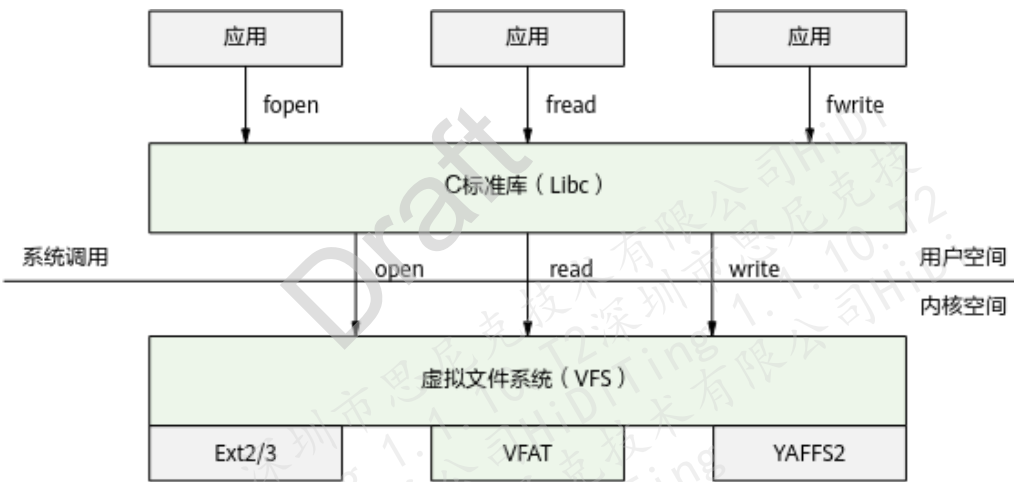
表 2-4 接口说明 9

Draft

1 概述

文件系统架构如图 1-1 所示。

图1-1 文件系统架构



文件系统包括 Libc、VFS、VFAT，用户应用可以通过 Libc 层文件接口使用文件系统。

虚拟文件系统 VFS (Virtual File System) 采用标准的 Unix 系统调用读写位于不同物理介质上的不同文件系统，为各类文件系统提供统一的操作界面和应用编程接口。VFS 向下兼容各底层存储介质和文件系统类型，实现对 open()、read()、write()等系统调用的直接调度而不用关心底层物理存储介质的差异。

虚拟文件分配表 VFAT (Virtual File Allocation Table) 采用 32 位的文件分配表，支持最大分区容量 128GB，最大文件 4GB，该文件系统提供了文件打开、关闭、读取、写入、删除等接口，用户可通过使用这些接口，对 eMMC 数据进行各类操作。

YAFFS (Yet Another Flash File System) 是一种专门针对闪存的文件系统，它是专为 NAND 类型的闪存存储器设计的。该文件系统提供了文件打开、关闭、读取、写入、删除等接口，用户可通过使用这些接口，对 NAND Flash 数据进行各类操作。

Draft

2 系统接口

2.1 VFS

2.2 YAFFS 文件系统

2.1 VFS

2.1.1 概述

虚拟文件系统 VFS 也可称为虚拟文件转换系统，是一个内核软件层。

VFS 有如下两个作用：

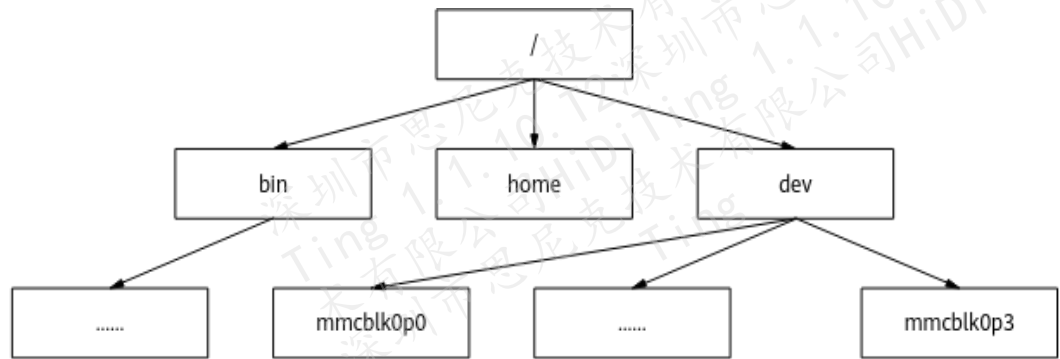
- 处理与 Unix 标准文件系统相关的所有系统调用。
- 为各种文件系统提供一个通用的接口。

2.1.2 开发流程

2.1.2.1 VFS 结构

虚拟文件系统 VFS 中的目录和文件按照目录树的方式组织。

图2-1 VFS 文件目录结构

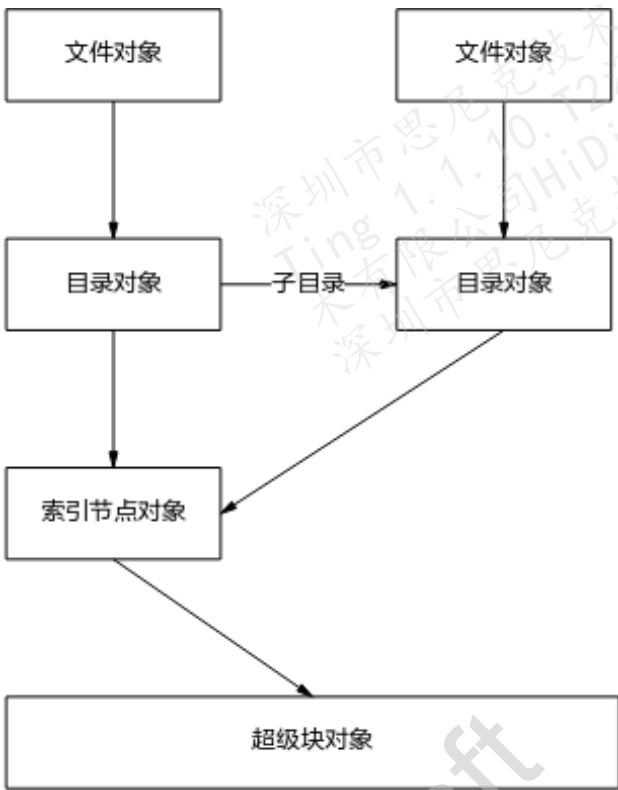


2.1.2.2 VFS 介绍

VFS 主要的四个对象如下：

- 超级块对象：代表一个特定的已挂载的文件系统。
- 索引节点对象：代表一个特定的文件项。
- 目录对象：代表一个特定的目录项。
- 文件对象：代表一个与进程相关的打开的文件。

图2-2 VFS 对象之间的关系



超级块对象

超级块对象（super_block）存储了描述特定文件系统的信息。通常超级块对象对应位于磁盘上特定扇区的文件系统超级块或文件系统控制块。

超级块对象由许多数据项组成，如下所示：

- 该文件系统所挂接的设备。
- 文件系统的基本块大小。
- 脏标志，表示超级块已经修改过，但还没有写回到磁盘。
- 文件系统类型。
- 标志，如只读标志。
- 指向文件系统根目录的指针。
- 打开文件的列表。
- 控制访问该文件系统的信号量。
- 操作超级块的函数指针数组的指针。

索引节点对象

一个索引节点对象 (inode) 与一个文件相关联。索引节点对象包含一个命名文件除该文件的文件名和该文件的实际数据内容外的所有信息，例如所有者、组、权限、文件的访问时间、数据长度和链接数等信息。

目录对象

目录对象 (directory entry) 是一个路径上的一个特定的组成。该组成是一个目录名或者是文件名，目录项对象为访问文件和目录提供了方便。

目录对象包括一个指向索引节点的指针和超级块，还包括一个指向父目录的指针和指向子目录的指针。

文件对象

文件对象 (file) 代表由进程打开的文件，存放打开文件与进程之间进行交互的有关信息，这些信息仅当进程访问文件期间存放在内核中。

文件对象由相应的 `open()` 系统调用创建，由 `close()` 系统调用撤销。

2.1.2.3 功能

标准 Posix 接口函数如表 2-1 所示。

表2-1 posix 接口函数列表

接口名称	接口描述
open	打开文件。
read	读取文件。
write	写入文件。
seek	移动文件读取位置。
stat	获取文件属性。
close	关闭打开的文件。
unlink	删除文件。

2.2 YAFFS 文件系统

2.2.1 概述

YAFFS (Yet Another Flash File System) 是一种适用于嵌入式系统的文件系统，主要用于闪存设备。而 NAND Flash 是一种闪存类型，也是一种非易失性存储器。YAFFS 文件系统用于管理 NAND Flash 上的数据。它可以在闪存中实现可靠的数据存储和访问，并提高数据存储的效率。YAFFS 文件系统提供了文件打开、关闭、读取、写入、删除等接口，用户可通过使用这些接口，对 NAND Flash 数据进行各类操作。

2.2.2 开发指导

NANDFlash 器件有不同的厂家，不同厂家在配置上有一些区别，在使用文件系统时要注意，如果使用的是华邦的 NANDFlash，需要在

“drivers/drivers/driver/nand_flash/nand/nandflash_config.c 和
bootloader/flashboot/brandy/nandflash/nandflash_config.c” 两个文件中将
get_nand_flash 方法替换成如下内容：

```
struct nand_driver_struct *get_nand_flash(void)
{
    return &g_w25n01g_driver;
}
```

YAFFS 文件系统的读写接口对上层业务开发人员是不可见的。在使用文件系统接口时，上层业务开发人员无需关注 YAFFS 文件系统的具体实现，只需要调用 VFS 层文件系统接口进行文件读写操作即可。具体的文件系统读写接口可在“[2.1.2.3 功能](#)”中查看。

YAFFS 文件系统的默认分区：

- 文件系统目前最大支持 128MB 文件存储。
- 磁盘默认被划分为 4 个分区，分区大小为剩余总容量的 10%、10%、10%、70%。如果需要调整分区比例，可以在“nandflash_config.c”（SDK 代码路径：
drivers/drivers/driver/nand_flash/nand/nandflash_config.c）文件中修改分区名称或分区大小的比例。分区名称与分区大小比例为——对应关系，如果需要增删需要修改 YF_VOLUMES_NUM 改成对应的数量。

```
#define YF_NAME_STRS "/system/", "/update/", "/music/", "/user/"
uint32_t yf_volume_percent[YF_VOLUMES_NUM] = { 10, 10, 10, 70 }; //system 10% , update
10% , music 10% , user 70%
```

表2-2 YAFFS 文件系统默认分区

设备节点	分区大小	挂载目录
/dev/nandblk0	10%	/system
/dev/nandblk1	10%	/music
/dev/nandblk2	10%	/update
/dev/nandblk3	70%	/user

2.2.2.1 接口说明

以下接口上层业务开发人员无需关注，仅供底层 YAFFS 文件系统集成人员参考使用。

表2-3 接口说明

头文件	接口名	描述
dpal_mtd.h	add_mtd_partition	添加 YAFFS 分区，该函数会自动为设备节点命名，对于 YAFFS，其命名规则为 “/dev/nandblk” 加分区号。
	delete_mtd_partition	删除已经卸载的分区。

- add_mtd_partition 添加 YAFFS 分区时，系统会自动对起始地址和分区大小根据设备 block 大小进行对齐。该函数有四个参数：
 - 第一个参数：介质，固定填写 “nand”。
 - 第二个参数：分区的起始地址，以 16 进制的形式传入。起始地址的取值范围取决于 NAND Flash 介质的存储空间大小。起始地址为 0x20000 的整数倍，包括地址 0。
 - 第三个参数：表示分区大小，以 16 进制的形式传入，分区大小应小于等于 NAND Flash 介质存储空间的大小，大小为 0x20000 的整数倍，例如 NAND Flash 的存储大小为 128MB，配置起始地址为 0 则配置的分区大小应小于等于 0x8000000。
 - 第四个参数：表示分区号，有效值为 0 ~ 19。
- delete_mtd_partition 函数有两个参数：
 - 第一个参数：分区号，与 add_mtd_partition()函数的第四个参数对应。

- 第二个参数：介质，固定填写“nand”。

表2-4 接口说明

头文件	接口名	描述
mount.h	mount	挂载分区
unmount.h	unmount	卸载分区

- mount()函数有五个参数：
 - 第一个参数：设备节点，这个参数需要与 add_mtd_partition()函数对应起来，设备节点“/dev/nandblk”为固定前缀，后面拼接 add_mtd_partition 的第四个参数分区号。
 - 第二个参数：挂载点，挂载点名称可自定义，类似于“/user/”或“/system/”这种根目录都可以。
 - 第三个参数：填写“yaffs”。
 - 第四个参数：分区的读写特性，填 0 可读可写。
 - 第五个参数：最后一个参数填写 NULL。
- unmount()函数有 1 个参数：

参数与 mount()的第二个参数相同。

2.2.2.2 开发流程

按如下步骤集成 YAFFS 文件系统，上层业务开发人员无需关注。

- 步骤 1 在“drivers/drivers/driver/nand_flash/nand/nandflash_config.c”添加如下代码，使能 NAND Flash。

```
struct nand_driver_struct * freertos_nandflash_driver = get_nand_flash();
PRINT("\n\nnandflash init start.\n\n");
nand_register(&nandflash_freertos_info);
freertos_nandflash_driver->nand_flash_init(freertos_default_buf, freertos_nandflash_driver->bytes_per_page + freertos_nandflash_driver->bytes_per_oob);
uint16_t id = freertos_nandflash_driver->check_id();
PRINT("\n\n nandflash chip_id : %x \n\n", id);
mtd_init_list();
add_mtd_list("nand", &nandflash_freertos_info);
```

- 步骤 2 调用 add_mtd_partition 创建分区，最多只能建立 20 个分区，分区号为 0~19。

创建分区的示例代码如下：

```
if (ret = add_mtd_partition("nand", 0x0, 0x1400000, 0) < 0) {  
    dprintf("add yaffs partition 0 failed, return %d\n", ret);  
}  
if (ret = add_mtd_partition("nand", 0x1400000, 0x2800000, 1) < 0) {  
    dprintf("add yaffs partition 1 failed, return %d\n", ret);  
}  
ret等于0 时创建成功
```

步骤 3 调用 mount 函数挂载分区。

```
ret = mount("/dev/nandblk0", "/users/", "yaffs", 0, NULL);  
if (ret) {dprintf("mount yaffs err %d\n", ret);}  
ret等于0时挂载成功
```

步骤 4 调用 unmount 函数卸载分区。

调用 unmount("/users/")函数卸载分区。

步骤 5 调用 delete_mtd_partition 函数删除分区。

调用 delete_mtd_partition 删除分区前要卸载分区。

```
ret = delete_mtd_partition(0, "nand");if (ret != 0)  
{ printf("delete yaffs error\n");} else { printf("delete yaffs ok\n");}
```

----结束

2.2.3 注意事项

须知

由于 NandFlash 器件支持内部 ECC (Error Correcting Code) 特性, 该特性在版本上需要使用宏 "ENABLE_ECC" 进行开启或者关闭, 当版本上该特性发生切换 (开启 -> 关闭或者关闭 -> 开启) 时, 由于 NandFlash 内部存储结构发生变化, 需要对文件系统进行格式化后再使用。

对于 YAFFS 的多分区特性, 分区起始地址可灵活配置, 但分区间地址不能重叠, 用户需要根据 Flash 使用情况合理配置空间, 防止与其它文件系统或其它不适合挂载 YFFAS 文件系统的空间发生冲突。

- 建议添加单个分区的 block 数量不少于 10 个 block。
- 建议分区使用前先进行擦除操作。

- 建议分区数应该小于等于 4 个，否则影响系统运行效率和内存占用。

Draft

3

镜像制作与烧录

3.1 概述

3.2 YAFFS 文件系统镜像制作与烧录

3.1 概述

YAFFS 的文件系统的镜像大小修改位置为

"\build\config\target_config\brandy\mk_fs_image\mkyaffs2tool.py"。

配置分区信息，请根据实际情况进行配置，添加格式为[分区路径， 分区大小]。

分区大小需要根据按照百分系统分配好后的 blocks 数进行计算，大小=blocks（文件块个数）*pages_per_block（每个文件块包含的页数）*bytes_per_page（每页包含字节数）。

置成 null 代表镜像大小根据资源文件大小自适应。

```
partition_list = [  
    [os.path.join(cur_dir, 'fs', 'user'), "null"]  
]
```

3.2 YAFFS 文件系统镜像制作与烧录

📖 说明

由于 YAFFS 文件系统镜像制作工具（mkyaffs2image）只有 Linux 版本，所以本文下述章节约束说明，YAFFS 文件系统镜像制作需要在 Linux 环境进行。

3.2.1 概述

制作 YAFFS 文件系统镜像前，我们要确认以下两点：

1. 使用的 Nandflash 器件参数，具体参数如下：

- blocks (文件块个数)
- pages_per_block (每个文件块包含的页数)
- bytes_per_page (每页包含的字节数)
- bytes_per_oob (每个 oob 包含的字节数)

例：Nandflash 的容量大小为 128MB，该器件有 1024 个 block，每个 block 有 64 个页，每页有 2048 个字节，每个 oob 区包含 64 个字节。

2. 当前版本的文件系统分区大小。

例：文件系统分四个区，system 分区，大小占 10%；update 分区，大小占 10%；music 分区，大小占 10%；user 分区，大小占 70%。

其中分区百分比主要用于计算 block 大小，system 分区大小占比 10%，即表示它占用的 block 数为 $1024 \times 10\% = 102.4$ ，system 分区 block 实际为 102，system 分区实际占用的字节数 $= 102 \times 64 \times 2048 = 13369344 \text{ Byte} = 0\text{xcc00000} \text{ Byte}$ ，其他分区也使用同样方式进行计算，可以得出分区字节大小。

须知

上述配置参数 (blocks、pages_per_block、bytes_per_page 和 bytes_per_oob) 必须和当前使用的 Nandflash 器件参数相同，否则制作的文件系统镜像会出现读写异常。

相关参数可查阅使用的 NandFlash 器件手册或咨询厂家获取。

3.2.2 开发指导

编译 YAFFS 文件系统镜像的脚本为 Python3 脚本，路径：

build\config\target_config\brandy\mk_fs_image\mkyaffs2tool.py。

制作文件系统镜像前，需要按照实际使用配置修改以下参数，配置参数都位于“mkyaffs2tool.py”脚本内头部，如下所示：

1. image_name：生成镜像的文件名，当前默认为“file”，生成文件名为 file.bin。
2. cfg_page_size：Nandflash 器件的 bytes_per_page 值，以实际器件为准。
3. cfg_oob_size：Nandflash 器件的 bytes_per_oob 值，以实际器件为准。

4. partition_list: YAFFS 文件系统的分区配置, 具体配置说明如下:

```
partition_list = [  
    [os.path.join(cur_dir, 'fs', 'system'), "0xcc00000"],  
    [os.path.join(cur_dir, 'fs', 'update'), "0xcc00000"],  
    [os.path.join(cur_dir, 'fs', 'music'), "0xcc00000"],  
    [os.path.join(cur_dir, 'fs', 'user'), "null"]  
]
```

每个分区都是 partition_list 中的一个元素, 而每个分区元素要包含制作的文件路径和分区大小信息, 文件路径为相对于 mkyaffs2tool.py 脚本所在位置的相对路径, 分区大小信息为十六进制。

如上示例配置, 文件系统镜像包含四个分区目录, 分别包含当前目录 fs 下的 system, update, music 和 user 四个子目录, 大小分别为 0xcc0000、0xcc0000、0xcc0000 和剩余空间。

需要注意的是文件系统的最后一个分区的大小可不指定, 直接置成 null 即可, 默认使用剩余空间。文件系统单分区时, 请直接置为 null 即可, 无需设置大小。

最后将需要制作的资源文件放到对应的打包文件路径中即可。

说明

当前生成的文件系统镜像大小是由 YAFFS 文件系统的分区配置和资源文件的大小决定的, 如果放在除最后一个分区的其他分区内的资源文件不足以填满这个分区, 那剩下空间将填充 0xFF 数据, 保证镜像文件内包含分区数据。只有最后一个分区只填充资源文件, 不会填充 0xFF 数据。所以, 如果想降低生成镜像大小, 首先除最后分区以外的其他分区的大小尽量缩小, 其次将资源文件都放在除最后分区的其他分区内。

举例:

1. 当前使用情况, 前面三个分区占 30%, 最后一个分区占 70%

资源文件放在前 3 个分区内, 则最终镜像大小 = 总空间 × 30%。

资源文件放在最后分区, 则最终镜像大小 = 总空间 × 30% + 资源文件大小。

2. 如果将前三个分区的大小修改为 20%, 最后一个分区大小占 80%

资源文件放在前面分区内, 则最终镜像大小 = 总空间 × 20%。

资源文件放在最后分区, 则最终镜像大小 = 总空间 × 20% + 资源文件大小。

3. 如果将 4 个分区修改成一个分区

则最终镜像大小 = 资源文件大小。

3.2.3 编译指导

支持烧录 YAFFS 文件系统镜像作为 SSB 功能特性，默认不打开。如需打开需要进行编译配置。

1. SSB 编译配置，需要在 SSB target 中添加宏：CFG_DRIVERS_NANDFLASH（SDK 代码路径：build/config/target_config/brandy/config.py）。

请注意，CFG_DRIVERS_NANDFLASH 宏和 CFG_DRIVERS_MMC 宏只能存在一个，根据需求进行设置。

如果 application 的 target 编译添加了 ENABLE_ECC 宏，则 SSB 编译也需要同时添加宏：ENABLE_ECC。

```
brandy-ssb': {
    'base_target_name': 'target_ssb_template_brandy',
    'board': 'evb',
    'defines': [
        "-:TARGET_CHIP_BRANDY-1", "-:BRANDY_CHIP_V100-1", "PRE_ASIC", "BRANDY_PRODUCT_EVB",
        "SSB_USE_PLL", "VERSION_STANDARD", "BUILD_APPLICATION_SSB", "LIBBOOTLOADER", "LIBCODELOADER",
        "LIBCPU_UTILS", "LIBAPP_VERSION", "LIBPANIC", "LIBCONNECTIVITY", "LIBERROR_CODE", "LIBSEC_RANDOM",
        "LIBCODELOADER_SSB", "LIBJLINK_LOAD", "LIBLIB_UTILS", "LIBBUILD_VERSION", "CFG_DRIVERS_NANDFLASH",
        "NO_TIMEOUT", "CONFIG_UART_BAUD_RATE_DIV_8", "SW_UART_DEBUG", "ENABLE_ECC",
    ],
    'ram_component': ['arch_port', 'hal_l2ram', 'sec_boot', 'libboundscheck',
        'dfx_exception', 'error_code', 'psram', 'osal', 'cmn_header', 'boot_nandflash'],
    'ram_component_set': ['pinctrl'],
    'target': 'ssb',
}
```

2. application 编译配置，在对应的 ACore target（SDK 代码路径：build/config/target_config/brandy/config.py）中将 fs_image 的开关置成 True，编译后生成的文件系统镜像位于 Acore 的 output 目录中与“application.bin”镜像同路径（SDK 代码路径：output/brandy/acore/brandy-target3/）。

```
#targets: freertos + lvgl + nand + mipi + psram + audio + media / 码型合封LVGL版本
brandy-targets': {
    'board': 'evb',
    'base_target_name': 'target_standard_brandy_application_freertos_template',
    'defines': [
        "PRE_ASIC", "BRANDY_PRODUCT_EVB4", "VERSION_STANDARD", "CONFIG_MTA_UPDATE_SUPPORT",
        "ALL_SOURCE", "SUPPORT_CXX", "I2C_SLAVE_REG_ADDR_4BYTE", "CONFIG_PSRAM_SUPPORT",
        "-:TARGET_CHIP_BRANDY-1", "-:BRANDY_CHIP_V100-1", "CFG_DRIVERS_NANDFLASH", "CONFIG_ZDIAG_IV_SUPPORT",
        "CONFIG_ZDIAG_AUDIO_PROC_SUPPORT", "CONFIG_ZDIAG_AUDIO_DUMP_SUPPORT", "CONFIG_ZDIAG_AUDIO_PROBE_SUPPORT", "CONFIG_SEA_PHS_SUPPORT",
        "SUPPORT_BLE1", "SUPPORT_BREDR", "ENABLE_LVGL", "SUPPORT_GPU_D3D8", "SUPPORT_GPU_GNUT", "SUPPORT_GPU_OPENVG",
        "SW_UART_DEBUG", "MEMORY_NO_CACHE", "SAVE_EXC_INFO", "FLASH_MEM_COPY", "MIPi_UTILS_SUPPORT",
        "CONFIG_LOW_POWER_TEST", "CONFIG_CDDIOPD_SUPPORT", "ENABLE_ECC",
    ],
    'ram_component': [
        'algorithm', 'dfx_port_brandy', 'dfx_update', 'dfx_nv', 'osal', 'arch_port', '-:testsuite',
        'lcd', 'gspi_display', 'partition', 'partition_brandy',
        'psram', 'hal_gpio', 'mipi_tx', 'non_os', 'touch',
        'ulp_aon', 'hal_l2ram',
        'x_dpall', 'x_vfs', 'x_disk', 'x_fat', 'drv_mmc', 'fs_yaffs2',
        'pm_service', 'pm_brandy', 'cmn_header', 'at_cmd', 'graphic_at_service', 'app_at_service',
        'audio_at_service', 'media_at_service', 'bt_manager_at_service',
        'audio_proc', 'audio_dump', 'audio_probe',
    ],
    'ram_component_set': ['bgh', 'media', 'gpu', 'graphic_lvgl', 'bgh_audio', 'update_app', 'dfx_set'],
    'dsp_version': 'max',
    'image_analysis': True,
    'packet': True,
    'fs_image': True,
}
```

3. 编译 SSB 和 application 以及版本包。

具体 target 以及版本包编译方法，请参考《HiDiTingV100 软件开发指南》中相关编译章节。

说明

编译补充：

由于 YAFFS 文件系统镜像制作工具（mkyaffs2image）只支持在 Linux 系统上使用，所以给出在 Window 系统上编译带有镜像的版本包方法。

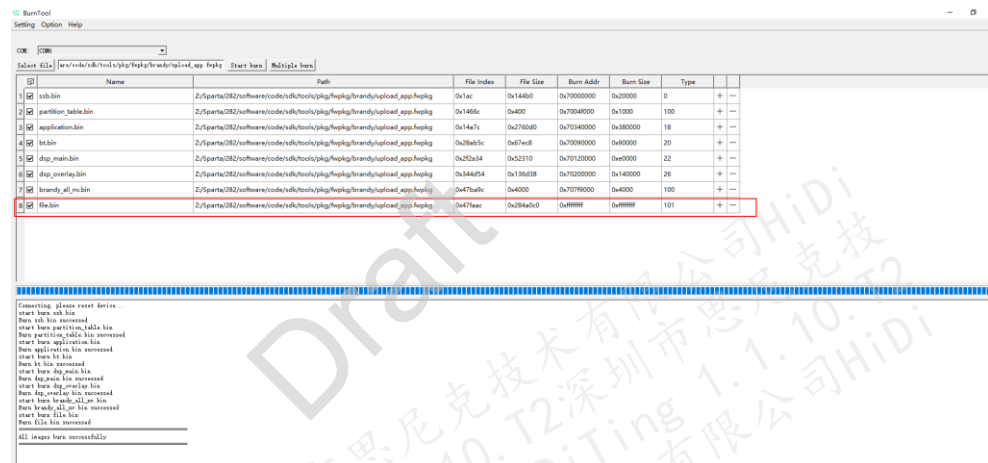
1. 镜像依旧需要在 Linux 系统上生成，跳转到
“build/config/target_config/brandy/mk_fs_image” 目录下，然后执行 “python mkyaffs2tool.py” 指令。
2. 将生成在 “build/config/target_config/brandy/mk_fs_image” 目录中的 “file.bin” 文件复制到 Window 系统上相同路径的目录中。
3. 按照上述编译指导配置相关参数，然后进行编译，即可将 “file.bin” 镜像打包到版本包中。

3.2.4 烧录指导

说明

由于 YAFFS 文件系统镜像较大，建议使用 UART 2M 及以上波特率进行烧录，减少烧录耗时。

1. 选择上述 SSB 和 application 编译后生成的 PKG 包，即可使用 BurnTool 工具进行版本整包烧录，其中 “file.bin” 为制作的文件系统镜像，默认勾选即可烧录。



2. 由于文件系统镜像 “file.bin” 实现在 SSB 阶段烧录进 NandFlash 器件，所以文件系统镜像 “file.bin” 需要与编译的 SSB 同时烧录。